VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

BCSE406L – NoSQL Databases | Fall Semester 2026–2027

Digital Assignment 2 (Review 2): Database Implementation & Application Progress Report

VERITAS

An Evidence-Aware, Temporal and Explainable NoSQL Database System for Evolving AI Debates

Full-Stack Document Database Implementation using MongoDB 7, FastAPI, PyMongo, React 19, React Flow, and Recharts

Submission Attribute	Official Record Details
Student Name	**Siddhesh Shinde**
Registration Number	**23BCE5048**
Project Role & Ownership	AI Analysis Pipeline, Multi-Factor Scoring Engine, Temporal History Service & Frontend Visualization Lead (Member 3)
Project Team Members	Naman Sharma (23BCE1683) — MongoDB Data Architect & Backend Lead Megh (23BCE1665) — FastAPI Routes, CRUD & QueryService Developer Siddhesh Shinde (23BCE5048) — Scoring Engine, Temporal History, AI Pipeline & React UI Lead
Course Code & Title	BCSE406L – NoSQL Databases
Slot & Class Number	Slot: D2+TD2 | Class ID: CH202670101354
Course Faculty	Dr. S. Geetha, School of Computer Science and Engineering, VIT Chennai
Submission Milestone	Digital Assignment 2 (Review 2 — Database Implementation & Query Progress)
GitHub Source Repository	https://github.com/siddheshshindeofficial/VERITAS-main

1. Executive Overview & System Architecture

1.1 Project Vision: Persistent Debate Knowledge Objects

Conventional conversational AI platforms treat complex analytical debates as transient chat transcripts. When a user asks a policy, clinical, or engineering question, standard large language model interfaces emit an unstructured block of prose that merges competing claims, hides source provenance, and discards the structural relationship between evidence and rebuttals once the session closes. **VERITAS** (*An Evidence-Aware, Temporal and Explainable Database for Evolving AI Debates*) resolves this limitation by re-architecting the debate itself as a persistent, queryable document graph stored inside MongoDB 7 (`veritas_db`).

Whenever a debate topic is initialized in VERITAS—such as *"Should AI be used for medical diagnosis?"* or *"Should AI-generated content require mandatory disclosure?"*—the platform creates a root document in the `debates` collection and links supporting (`PRO`) and opposing (`CON`) arguments in the `claims` collection. Each claim is connected via MongoDB `ObjectId` references to empirical or institutional records in `evidence`, bibliographic metadata in `sources`, targeted rebuttals in `counterarguments`, pairwise conflicts in `contradictions`, and chronological state transitions in `debate_history`.

The complete source code, seed fixtures, index builders, and test suites are maintained in our public GitHub repository at https://github.com/siddheshshindeofficial/VERITAS-main. The application pairs a Python 3.12 and FastAPI backend (using PyMongo 4 for native document operations) with a responsive React 19 and TypeScript frontend featuring live Recharts telemetry, React Flow argument graphs, and an interactive Query Explorer.

1.2 Justification for Selecting MongoDB & the Document Data Model

Representing an evolving multi-party debate requires a storage engine that handles hierarchical state, polymorphic metadata, and cross-entity analytical aggregations without rigid relational join overhead. MongoDB 7 was selected as the primary NoSQL database engine for five engineering reasons:

- **Co-located Read Optimization via Embedding:** Every time a debate workspace or dashboard card loads, the interface requires the root question alongside its latest synthesis metrics (`pro_score`, `con_score`, `confidence`, and `uncertainty`). Embedding `current_synthesis` and `metadata` inside the `debates` document allows single-document retrieval in `O(1)` disk seeks.
- **Independent Lifecycle & Cardinality via ObjectId Referencing:** Claims, evidence items, counterarguments, and pairwise contradictions grow asynchronously and are queried independently for ranking and filtering. Storing them as top-level collections linked by `ObjectId` prevents unbounded array growth inside the root debate document.
- **Polymorphic Provenance & Schema Flexibility:** Documents created manually by a researcher, ingested via automated seed pipelines, or synthesized through the AI analysis workflow carry distinct provenance attributes (such as `generated_by`, `source_context`, or `strength_explanation`). MongoDB's flexible BSON schema accommodates these variations natively without schema migrations.
- **Append-Only Temporal Auditing:** Whenever claims are added, evidence weights change, or contradictions are flagged, the temporal service appends an immutable event snapshot to `debate_history`, enabling time-series reconstruction of confidence shifts.

- **Expressive Aggregation & Lookup Pipelines:** MongoDB's native aggregation framework (`$match`, `$group`, `$lookup`, `$addFields`, `$map`, `$cond`, and `$sort`) executes multi-collection evidence weighting and domain-level statistical rollups directly inside the database engine.

1.3 Three-Tier Architecture & Technology Stack

VERITAS is structured into three decoupled layers: (1) a client presentation tier built with React 19, Vite 8, TypeScript, Tailwind CSS 4, TanStack Query, React Flow, and Recharts; (2) an application and domain service tier powered by FastAPI 0.115 and Pydantic v2; and (3) a persistent document storage tier running MongoDB 7 inside a Docker Compose container (`veritas-mongo` on port `27017`). Figure 1.1 illustrates the request flow across these tiers.

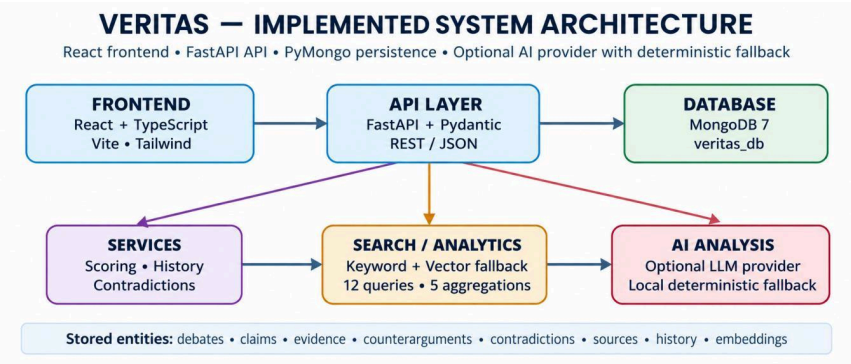


Figure 1.1: VERITAS three-tier system architecture connecting React 19, FastAPI/PyMongo services, and MongoDB 7.

System Layer	Core Technologies	Architectural Role in VERITAS
Frontend UI	React 19, TypeScript, Vite 8, Tailwind CSS 4, TanStack Query	Renders interactive debate workspaces, CRUD forms, live Query Explorer (Q1–Q12), Admin index monitor, and Database Health telemetry.
Graph & Charts	React Flow, Recharts, Lucide Icons, React Hook Form, Zod	Visualizes claim-evidence-contradiction node graphs, PRO/CON confidence distributions, source breakdowns, and temporal evolution curves.
Backend API	Python 3.12, FastAPI 0.115, Pydantic 2, Uvicorn	Exposes validated REST/JSON endpoints, serializes BSON `ObjectId` and `datetime` objects, and orchestrates domain services.
NoSQL Database	MongoDB 7 (Docker `mongo:7`), PyMongo 4 Driver	Persists 8 active collections in `veritas_db`, enforces 8 custom compound/single-field indexes, and executes analytical pipelines.
AI & Scoring	Custom Python Scoring Engine + OpenAI-Compatible Adapter	Computes deterministic evidence/claim scores, Jaccard heuristic contradiction detection, and LLM synthesis with deterministic local fallback.
Verification	Pytest Automated Test Suite, Swagger UI (OpenAPI 3.1)	Executes 11 end-to-end API and database verification tests against live MongoDB collections and provides interactive endpoint testing.

1.4 Repository Structure & Local Execution

The project repository (https://github.com/siddheshshindeofficial/VERITAS-main) separates backend database logic, indexing/seeding scripts, automated tests, and frontend views into clean modular directories:

```
VERITAS/
├── backend/
│   ├── app/
│   │   ├── main.py                  # FastAPI entry point, CORS middleware, router
registration
│   │   ├── config.py                # Environment configuration (MONGODB_URI, MONGODB_DB)
│   │   ├── database.py              # Shared PyMongo client, ping check, live collection
counters
│   │   ├── routes/                  # REST controllers: debates, claims, evidence, sources,
ai, health
│   │   ├── schemas/                 # Pydantic v2 request/response validation schemas
│   │   ├── services/                # scoring.py, claim_strength.py,
contradiction_engine.py, temporal.py
│   │   ├── analytics/               # queries.py (Q1-Q12) and service.py (AGG-1 to AGG-5)
│   │   ├── agents/                  # debate_pipeline.py, llm_provider.py, fallback.py
│   │   └── search/                  # Keyword matching and cosine vector-similarity search
plumbing
│   ├── scripts/                     # indexes.py, seed.py, counts.py, query_examples.js
│   └── tests/test_api.py            # 11 automated Pytest integration tests
├── frontend/src/                    # React 19 pages (Dashboard, Workspace, QueryExplorer,
Admin, Health)
└── docker-compose.yml               # MongoDB 7 container definition on port 27017
```

To launch the complete stack locally, the MongoDB 7 container is started via Docker Compose, indexes and seed fixtures are initialized via Python scripts, and the Uvicorn API server and Vite frontend server are started in parallel:

```
# 1. Start MongoDB 7 container and initialize indexes + dataset
docker compose up -d mongodb
cd backend && .\venv\Scripts\activate
python scripts/indexes.py && python scripts/seed.py
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# 2. Launch React 19 development server (http://127.0.0.1:5173)
cd frontend && npm run dev -- --host 127.0.0.1
```

2. DA2 Review 2 Technical Implementation & Rubric Mapping

2.1 Finalized Problem Statement & Research Gap (Rubric A1)

When evaluating controversial multi-domain questions, analysts need more than a static summary: they require a traceable audit trail showing how individual supporting or opposing claims are backed by primary evidence, where claims directly contradict one another, and how overall confidence evolves as new evidence arrives. Existing conversational interfaces fail to persist these relationships as first-class queryable records.

VERITAS bridges this gap by modeling every debate as a multi-collection NoSQL document graph. Rather than delegating numerical scoring to opaque language model probabilities, VERITAS implements an explicit, reproducible mathematical scoring pipeline in Python (`backend/app/services/scoring.py`) and stores every intermediate factor directly inside MongoDB documents.

2.2 Final NoSQL Data Model & Hybrid Design Decisions (Rubric A2)

The `veritas_db` database comprises nine defined collections—eight actively populated during standard operation and one reserved collection (`audit_logs`) for administrative security logging. Figure 2.1 illustrates the conceptual document model and cross-collection reference graph.

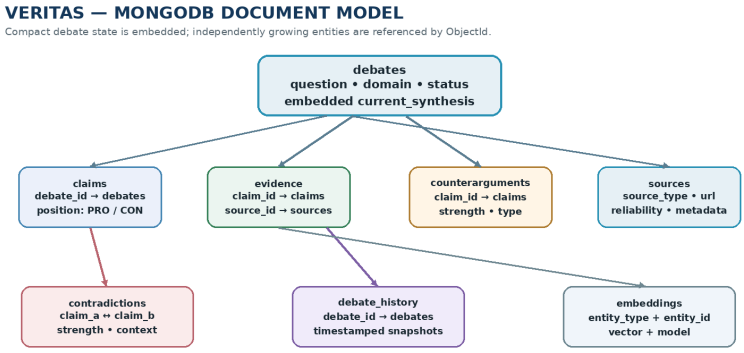


Figure 2.1: VERITAS conceptual document model illustrating embedded synthesis objects and ObjectId references.

Collection Name	Modeling Strategy	Primary Fields, Relationships & Access Role
`debates`	Root Document + Embedded Synthesis	Stores `question`, `domain`, `status`, `created_at`, embedded `current_synthesis` (`summary`, `confidence`, `pro_score`, `con_score`, `uncertainty`), and `metadata`.
`claims`	Referenced (`debate_id`)	Stores `position` (`PRO` | `CON`), `text`, `confidence`, `status`, `tags[]`, navigation caches `evidence_ids[]` and `counterargument_ids[]`, and `provenance`.
`evidence`	Referenced (`claim_id`, `source_id`)	Stores `title`, `content`, `source_type`, `supports` (boolean), `relevance`, `reliability_score`, `independence_score`, `recency_score`, and computed `evidence_score`.
`counterarguments`	Referenced (`claim_id`)	Stores targeted challenges to a claim: `claim_id`, `text`, `strength`, `type` (`rebuttal` | `qualification` | `counterclaim`), and `generated_by`.
`contradictions`	Pairwise Reference (`claim_a`, `claim_b`)	Links conflicting PRO and CON claims with `strength`, `type` (`direct_contradiction` | `contextual_disagreement` | `potential_conflict`), `explanation`, and `context_dimensions`.
`sources`	Reusable Referenced Catalogue	Stores normalized bibliographic records: `title`, `authors[]`, `publisher`, `source_type`, `url`, `published_at`, and baseline `reliability`.
`debate_history`	Append-Only Referenced Stream	Captures chronological state snapshots per `debate_id`: `timestamp`, `pro_score`, `con_score`, `confidence`, `event_type`, and `change_summary`.
`embeddings`	Polymorphic Reference	Stores vector arrays (`embedding[]`) linked via `entity_type` (`claim` | `evidence` | `counterargument`) and `entity_id` for similarity search.

Collection Name	Modeling Strategy	Primary Fields, Relationships & Access Role
`audit_logs`	Reserved System Trail	Defined in database health telemetry for system-level compliance auditing (count: `0` in local academic deployment).

2.3 Database Implementation & Document Schemas (Rubric A3 & A4)

Database connectivity is managed in `backend/app/database.py`, which initializes a singleton `MongoClient(settings.MONGODB_URI)`, verifies server responsiveness via `admin.command("ping")`, and queries live collection counts dynamically via `count_documents({})`. Below are the concrete BSON document structures stored in `veritas_db` (illustrated using the seeded Healthcare debate `6aaeee47bba2e3c0b46e2e74`):

```
// 1. debates collection document (_id: ObjectId("6aaeee47bba2e3c0b46e2e74"))
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e74"),
  "question": "Should AI be used for medical diagnosis?",
  "domain": "Healthcare",
  "description": "Examining AI-assisted diagnostic tools in clinical settings.",
  "status": "active",
  "current_synthesis": {
    "summary": "Balanced multi-source synthesis across clinical validation and regulatory
risk.",
    "confidence": 0.6175,
    "pro_score": 0.625,
    "con_score": 0.61,
    "uncertainty": 0.985
  },
  "metadata": { "created_by": "seed_script", "language": "en", "development_fixture": true }
}

// 2. claims collection document (_id: ObjectId("6aaeee47bba2e3c0b46e2e76"))
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e76"),
  "debate_id": ObjectId("6aaeee47bba2e3c0b46e2e74"),
  "position": "PRO",
  "text": "Evidence suggests significant benefits in Healthcare contexts when properly
validated.",
  "confidence": 0.6,
  "status": "active",
  "tags": ["healthcare"],
  "evidence_ids": [ObjectId("6aaeee47bba2e3c0b46e2e78"),
ObjectId("6aaeee47bba2e3c0b46e2e79")],
  "counterargument_ids": [ObjectId("6aaeee47bba2e3c0b46e2e7a")],
  "provenance": { "generated_by": "retrieval", "source_context": "seed_script" }
}

// 3. evidence collection document (_id: ObjectId("6aaeee47bba2e3c0b46e2e78"))
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e78"),
  "claim_id": ObjectId("6aaeee47bba2e3c0b46e2e76"),
  "source_id": ObjectId("6aaeee47bba2e3c0b46e2e5e"),
  "title": "Clinical validation study for AI diagnostic triage",
  "source_type": "research_paper",
  "supports": true,
  "relevance": 0.55,
  "reliability_score": 0.5,
```

```
  "independence_score": 0.55,
  "recency_score": 0.6,
  "evidence_score": 0.0908
}
```

The evidence score is computed deterministically in `backend/app/services/scoring.py` as the clamped product of four normalized dimensions: `Evidence Score = round(clip(reliability) * clip(relevance) * clip(independence) * clip(recency), 4)`. For the document above, `0.50 × 0.55 × 0.55 × 0.60 = 0.09075`, which rounds to `0.0908`.

Across our Review 2 baseline seed and interactive test sessions, `veritas_db` tracks five core domain debates (*Healthcare*, *Transportation*, *Education*, *Environment*, and *Technology*), scaling from 238 initial baseline documents (5 debates, 24 claims, 48 evidence records, 24 counterarguments, 13 contradictions, 22 sources, 30 history events, and 72 embeddings) to 507 total documents (11 debates, 52 claims, 100 evidence items, 52 counterarguments, 29 contradictions, 44 sources, 67 history snapshots, and 152 embeddings) after running extended multi-debate analysis workflows.

2.4 CRUD Operations & Application-Database Mutations (Rubric A5)

Complete Create, Read, Update, and Delete (CRUD) workflows are implemented across `backend/app/routes/debates.py`, `claims.py`, `evidence.py`, and `sources.py`:

- **CREATE (`POST /api/debates`, `POST /api/claims`, `POST /api/evidence`):** Validates incoming payloads via Pydantic v2, inserts the BSON document into MongoDB, recalculates parent claim/debate scores when claims or evidence are added, and appends a `debate_created` or `claim_added` snapshot to `debate_history`.
- **READ (`GET /api/debates`, `GET /api/debates/{id}`, `GET /api/claims/debate/{id}`):** Supports domain/status filtering, pagination, and single-document lookup by `ObjectId`.
- **UPDATE (`PUT /api/debates/{id}`, `PUT /api/claims/{id}`):** Executes atomic `$set` mutations (for example, transitioning a debate's `status` to `"completed"` and updating `updated_at`).
- **DELETE (`DELETE /api/debates/{id}`):** Executes application-level cascading cleanup across debate-owned records (removing associated documents from `claims`, `evidence`, `counterarguments`, and `debate_history` while preserving shared `sources`).

```
// Equivalent mongosh CRUD operations executed by the FastAPI routes:
db.debates.insertOne({ question: "Should AI be used for medical diagnosis?", domain:
"Healthcare", status: "active", created_at: new Date() })
db.debates.find({ domain: "Technology", status: "active" }).sort({ created_at: -1 })
db.debates.updateOne({ _id: ObjectId("6aaef7c0e6ab4eed455b0053") }, { $set: { status:
"completed", updated_at: new Date() } })
db.debates.deleteOne({ _id: ObjectId("6aaef7c0e6ab4eed455b0053") })
```

2.5 Twelve Project-Specific MongoDB Queries (Q1–Q12) (Rubric A6)

Twelve analytical queries are implemented in `backend/app/analytics/queries.py`, exposed via `GET /api/analytics/queries/{q1..q12}`, and executable live in the browser through the Query Explorer (`/query-explorer`). Table 2.2 summarizes all 12 queries, and the key aggregation pipelines are detailed below.

ID	Query / Analytical Operation	MongoDB Operators Used	Observed Output Summary
Q1	Fetch all debates, newest first	`find()`, `sort({created_at: -1})`, `limit(50)`	Returns active debates ordered chronologically across all 5 domains.
Q2	Retrieve claims for a selected debate	`find({debate_id: ObjectId})`, `sort({confidence: -1})`	Returns PRO and CON claims ranked by confidence (e.g., 4 claims for Healthcare).
Q3	Compare PRO vs. CON average confidence	`$match`, `$group`, `$avg`, `$sum`	Healthcare: `PRO` avg `0.625` (2 claims), `CON` avg `0.610` (2 claims).
Q4	Strongest supporting evidence for a claim	`find({claim_id, supports: true})`, `sort({evidence_score: -1})`	Returns top supporting evidence ranked by multiplicative `evidence_score`.
Q5	Strongest opposing evidence for a claim	`find({claim_id, supports: false})`, `sort({evidence_score: -1})`	Returns highest-weighted opposing evidence challenging the target claim.
Q6	High-strength contradictions filter	`find({strength: {$gte: threshold}})`, `sort({strength: -1})`	Filters conflict pairs by numeric threshold (`0.35` for baseline conflicts).
Q7	Average evidence reliability by source type	`$group` on `$source_type`, `$avg`, `$sum`, `$sort`	Ranks `dataset` (0.80) > `news` (0.70) > `institutional` (0.617) > `research_paper` (0.544) > `web` (0.50).
Q8	Debate temporal evolution timeline	`find({debate_id})`, `sort({timestamp: 1})`	Returns ordered event snapshots feeding the Recharts Temporal Evolution view.
Q9	Low-confidence active claims scan	`find({confidence: {$lt: 0.4}, status: "active"})`	Identifies weak claims requiring additional evidence validation.
Q10	Global ranking of active claims	`find({status: {$ne: "superseded"}})`, `sort({confidence: -1})`	Returns top 20 highest-confidence claims across the entire database.
Q11	Evidence-weighted claim score via `$lookup`	`$match`, `$lookup`, `$addFields`, `$map`, `$cond`, `$sum`, `$sort`	Joins `claims` with `evidence` and computes net weighted support score.

ID	Query / Analytical Operation	MongoDB Operators Used	Observed Output Summary
Q12	Domain-level debate synthesis statistics	`$group` on `$domain`, `$sum`, `$avg`, `$sort`	Aggregates debate count, mean confidence, mean PRO, and mean CON per domain.

```
// Q3: Compare PRO vs CON average confidence within a debate
db.claims.aggregate([
  { $match: { debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74"), status: { $ne: "superseded" }
} },
  { $group: { _id: "$position", avg_confidence: { $avg: "$confidence" }, count: { $sum: 1 } }
}
])

// Q7: Average evidence reliability grouped by source_type
db.evidence.aggregate([
  { $group: { _id: "$source_type", avg_reliability: { $avg: "$reliability_score" }, count: {
$sum: 1 } } },
  { $sort: { avg_reliability: -1 } }
])

// Q11: Cross-collection $lookup joining claims and evidence for net weighted scoring
db.claims.aggregate([
  { $match: { debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74") } },
  { $lookup: { from: "evidence", localField: "_id", foreignField: "claim_id", as:
"evidence_items" } },
  { $addFields: {
      weighted_score: {
        $sum: { $map: {
          input: "$evidence_items", as: "e",
          in: { $cond: ["$$e.supports", "$$e.evidence_score", { $multiply:
["$$e.evidence_score", -0.5] }] }
        }}
      }
  }},
  { $sort: { weighted_score: -1 } }
])

// Q12: Domain-level synthesis rollup
db.debates.aggregate([
  { $group: {
      _id: "$domain", debate_count: { $sum: 1 },
      avg_confidence: { $avg: "$current_synthesis.confidence" },
      avg_pro: { $avg: "$current_synthesis.pro_score" },
      avg_con: { $avg: "$current_synthesis.con_score" }
  }},
  { $sort: { debate_count: -1 } }
])
```

2.6 Indexing Strategy, Aggregations & Individual Contributions (Rubric A7–A12)

Eight custom application indexes (plus the default `_id_` primary index on each collection, totaling 16 active indexes across the 8 populated collections) are created via `backend/scripts/indexes.py` to align with the exact equality-filter and sort predicates executed by the REST API:

- `debates`: Compound index `domain_1_created_at_-1` (`domain ASC, created_at DESC`) accelerating domain filtering and chronological listing.

- `claims`: Compound index `debate_id_1_position_1_confidence_-1` serving the workspace PRO/CON split panels and Q2/Q3 lookups without in-memory collection scans.
- `evidence`: Four-key compound index `claim_id_1_supports_1_reliability_score_-1_relevance_-1` optimizing Q4 and Q5 supporting/opposing evidence retrieval.
- `contradictions`: Single-field descending index `strength_-1` supporting threshold range scans (Q6).
- `debate_history`: Compound index `debate_id_1_timestamp_1` serving chronological timeline rendering (Q8).
- `sources`, `embeddings`, and `counterarguments`: Indexed on `(source_type ASC, published_at DESC)`, `(entity_type ASC, entity_id ASC)`, and `(claim_id ASC)` respectively.

In addition to the 12 named queries, `backend/app/analytics/service.py` exposes five dedicated aggregation pipelines on the Admin Analytics dashboard: **AGG-1** (PRO vs. CON confidence distribution), **AGG-2** (Evidence quality and reliability by source type), **AGG-3** (Evidence coverage count and mean score per claim), **AGG-4** (Top 10 highest-confidence claims across domains), and **AGG-5** (Contradiction category distribution and average conflict strength).

Team Member	Register No.	Implemented Engineering Modules (DA2 Review 2 Scope)
Naman Sharma	23BCE1683	MongoDB 7 container architecture, `veritas_db` collection design, hybrid embedding vs. referencing specification, seed script (`seed.py`), and index definitions (`indexes.py`).
Megh	23BCE1665	FastAPI CRUD routers (debates, claims, evidence, sources), Pydantic v2 validation schemas, `QueryService` (Q1–Q12), aggregation service (AGG-1 to AGG-5), and 11 Pytest API tests.
Siddhesh Shinde	**23BCE5048**	**Multi-factor evidence & claim scoring engine (`scoring.py`, `claim_strength.py`), heuristic contradiction detector (`contradiction_engine.py`), append-only temporal history service (`temporal.py`), AI analysis & deterministic fallback pipeline (`agents/`), and complete React 19 UI (Dashboard, Debate Workspace, React Flow Debate Graph, Recharts Temporal Evolution, Query Explorer, Admin & Health views).**

3. Implementation Evidence & Live Execution Screenshots

3.1 MongoDB Compass, Docker Container & Stored BSON Documents (Rubric B1)

The screenshots below verify the running `veritas-mongo` Docker container (MongoDB 7 on port `27017`) and the live collections and BSON documents inspected inside MongoDB Compass. In Figure 3.1, the Docker container logs confirm clean initialization of the WiredTiger storage engine and active client listener connections on port 27017.

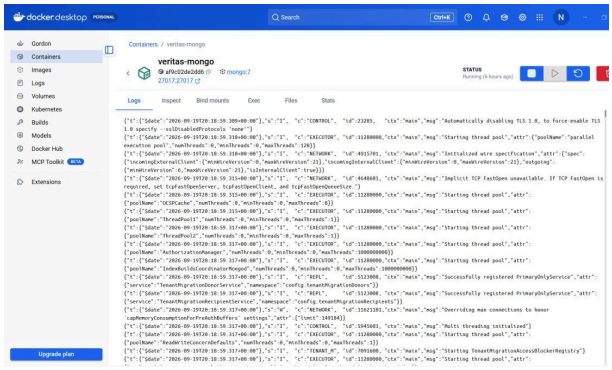


Figure 3.1: Docker Desktop running the veritas-mongo (mongo:7) container bound to localhost:27017.

Connecting MongoDB Compass to `mongodb://localhost:27017` reveals all eight populated collections inside `veritas_db` (Figure 3.2), along with their storage footprints, average document sizes, and the 16 active indexes (two per collection: the primary _id_ index plus the custom access-pattern index).

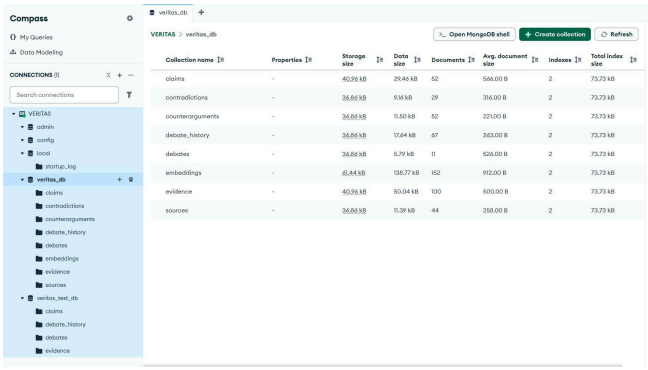


Figure 3.2: MongoDB Compass showing the 8 populated collections in veritas_db with storage sizes, document counts, and 2 indexes per collection (16 total).

Inspecting the `debates` collection in MongoDB Compass (Figure 3.3) confirms that each root debate document embeds its `current_synthesis` object and `metadata` subdocument directly, allowing the dashboard and workspace header to render without secondary lookups.

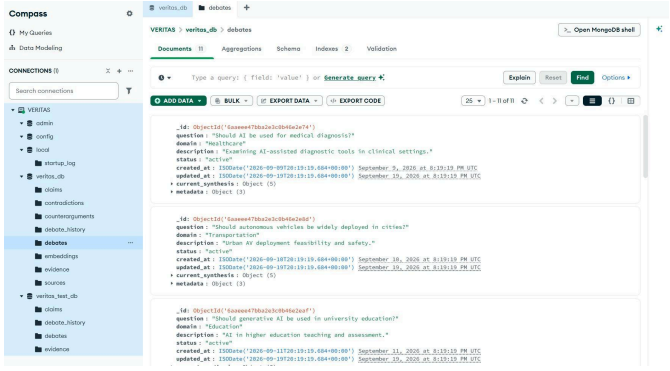


Figure 3.3: MongoDB Compass inspecting root debate documents in veritas_db.debates with embedded current_synthesis and metadata subdocuments.

Similarly, Figures 3.4 and 3.5 show the referenced documents inside `veritas_db.claims` and `veritas_db.evidence`. Each claim stores its parent `debate_id`, stance (`PRO` or `CON`), confidence score, and navigation arrays (`evidence_ids`, `counterargument_ids`), while each evidence document records its parent `claim_id`, bibliographic `source_id`, four normalized evaluation dimensions, and the computed `evidence_score`.

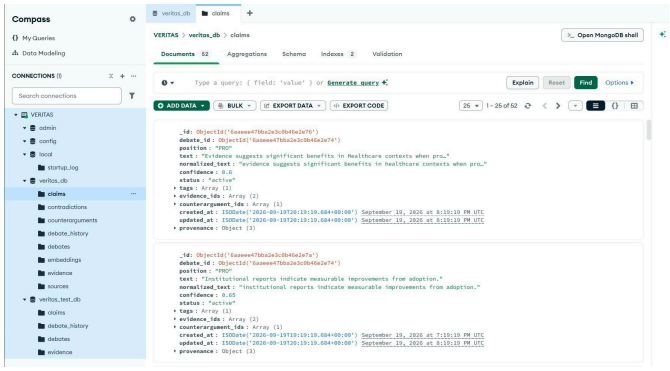


Figure 3.4: MongoDB Compass showing PRO and CON claim documents in veritas_db.claims linked via debate_id and evidence_ids.

In Figure 3.5, the `veritas_db.evidence` collection demonstrates how supporting and opposing evidence items store their normalized evaluation factors (`relevance`, `reliability_score`, `independence_score`, and `recency_score`) alongside the precomputed `evidence_score` and ISO-8601 publication timestamps.

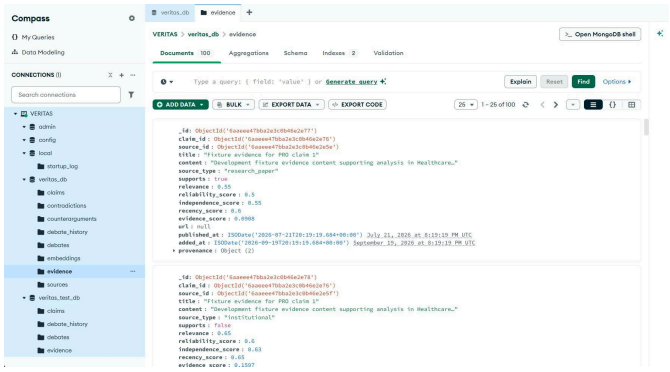


Figure 3.5: MongoDB Compass displaying evidence documents in veritas_db.evidence with multi-factor reliability, relevance, independence, recency, and computed evidence_score fields.

3.2 Query Explorer Execution Evidence (Q1–Q12) & Index Verification (Rubric B2)

The React Query Explorer (`/query-explorer`) executes the predefined MongoDB queries against `veritas_db` and displays execution latency in milliseconds alongside the raw BSON/JSON payload. Figure 3.6 demonstrates Query Q2 retrieving all active claims for a selected debate ordered by descending confidence in `17.23 ms`.

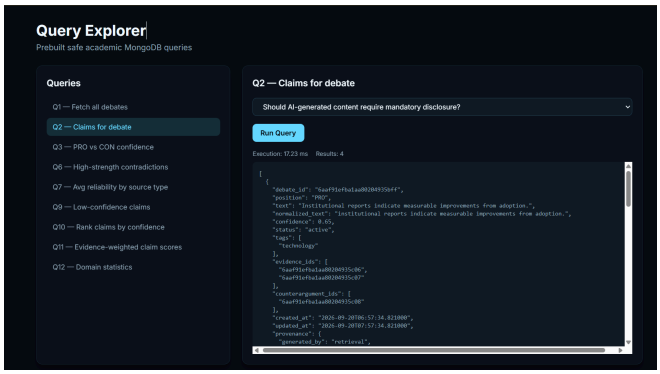


Figure 3.6: Query Explorer executing Q2 (Claims for a selected debate) in 17.23 ms, returning 4 linked PRO/CON claim documents.

Figure 3.7 shows the multi-stage `$group` and `$sort` aggregation pipeline for Query Q7, which computes the mean reliability score and total document count across each evidence source category (`dataset`, `news`, `institutional`, `user_provided`, and `research_paper`) in `7.69 ms`.

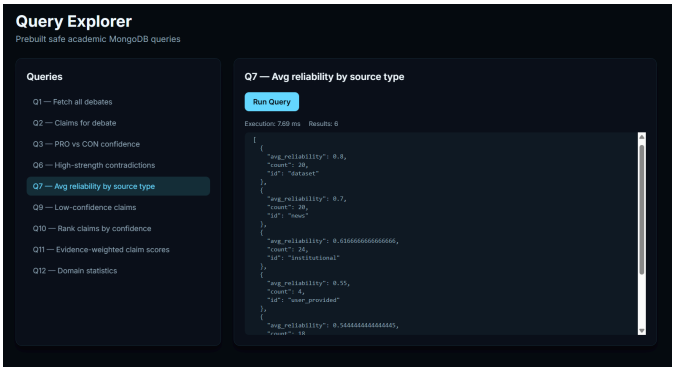


Figure 3.7: Query Explorer executing Q7 (Average evidence reliability grouped by source_type) in 7.69 ms across source categories.

Figure 3.8 demonstrates Query Q11, our most complex cross-collection aggregation pipeline. It executes a `$lookup` join from `claims` to `evidence`, applies a conditional `$map` and `$cond` expression inside `$addFields` to weight supporting evidence positively (`+1.0 × evidence_score`) and opposing evidence negatively (`-0.5 × evidence_score`), and sorts claims by net `weighted_score` in `20.6 ms`. Figure 3.9 verifies the underlying custom indexes on the Admin page.

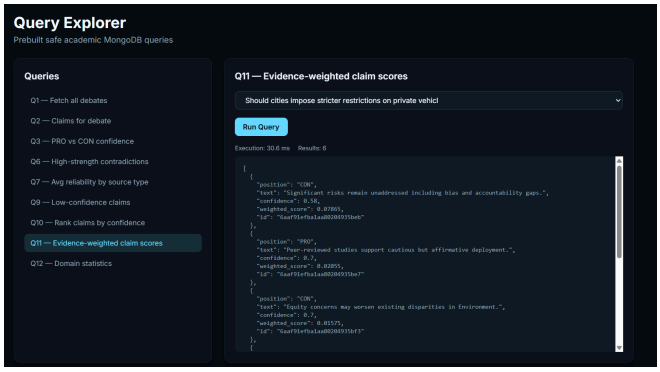


Figure 3.8: Query Explorer executing Q11 (Cross-collection $lookup evidence-weighted claim score pipeline) in 20.6 ms.

To verify that all queries execute via indexed B-tree scans rather than full collection scans, the Admin page (Figure 3.9) queries MongoDB's index catalogue and displays the exact key specifications for all 16 active indexes across the eight populated collections.



Figure 3.9: VERITAS Admin view displaying all 16 active MongoDB indexes across the 8 collections.

3.3 React 19 Application Views, Debate Graph & Temporal Charts (Rubric B3)

Figures 3.10 through 3.16 demonstrate the frontend views built by Siddhesh Shinde (`23BCE5048`). The landing page (Figure 3.10) provides direct entry points to start or explore debates, while the live Analytics Dashboard (Figure 3.11) renders real-time collection totals, domain bar charts, PRO/CON balance histograms, and source distribution pie charts directly from FastAPI aggregation endpoints.

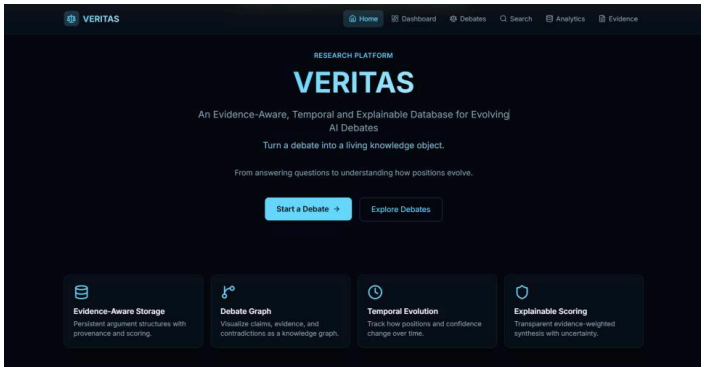


Figure 3.10: VERITAS landing page introducing Evidence-Aware Storage, Debate Graph, Temporal Evolution, and Explainable Scoring.

Navigating to `/dashboard` (Figure 3.11) executes parallel aggregation requests via TanStack Query to render real-time document totals (11 debates, 52 claims, 100 evidence items, 29 contradictions), domain distribution bar charts, PRO/CON stance histograms, and evidence source breakdowns.

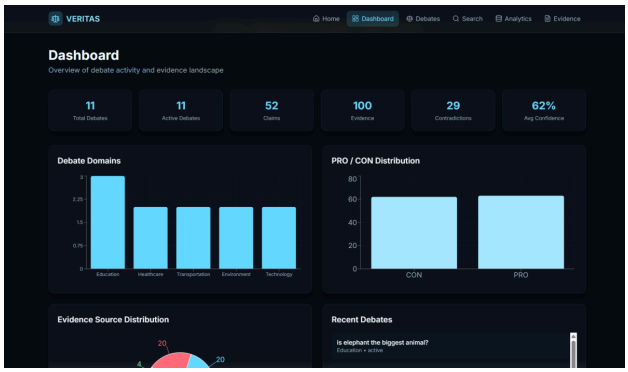


Figure 3.11: Live Analytics Dashboard displaying collection counters, domain distribution, PRO/CON balance, and evidence source breakdown.

The Explore Debates view (Figure 3.12) supports domain and status filtering over `GET /api/debates`. Selecting a debate opens the interactive Debate Workspace (Figure 3.13), which displays side-by-side PRO and CON claim cards alongside the embedded React Flow argument graph.

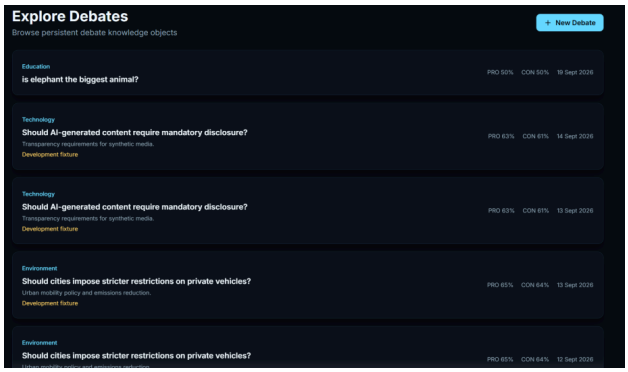


Figure 3.12: Explore Debates interface loading paginated debate records from GET /api/debates with PRO/CON confidence badges.

Clicking any debate card in the browser navigates to `/debates/:id` (Figure 3.13), where the workspace loads the root debate document alongside its indexed PRO and CON claims and constructs the interactive React Flow node-link graph.



Figure 3.13: Debate Workspace displaying side-by-side PRO (63%) and CON (61%) claim panels and the interactive React Flow Debate Graph.

Within the workspace, the React Flow Debate Graph and Evidence Explorer (Figure 3.14) allow users to inspect directed edges (`DERIVED_FROM`, `SUPPORTS`, `OPPOSES`, and `ATTACKS`) and filter evidence cards by stance and source category. Below the graph, the Recharts Temporal Evolution panel (Figure 3.15) plots historical PRO, CON, and confidence trajectories from `debate_history` alongside the Evidence-Weighted Synthesis summary. Figure 3.16 confirms live database connectivity and `2.66 ms` ping latency on the Database Health view.

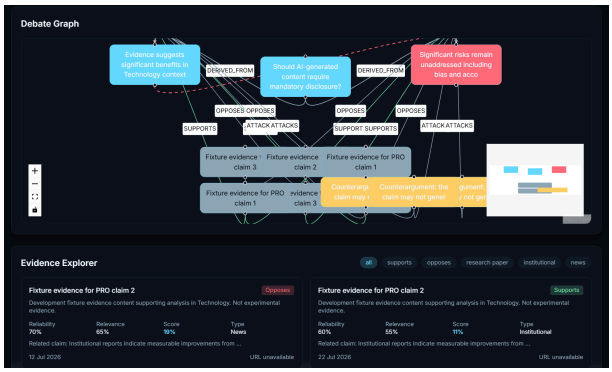


Figure 3.14: Detailed React Flow Debate Graph (DERIVED_FROM, SUPPORTS, OPPOSES, ATTACKS edges) and Evidence Explorer filter cards.

Scrolling down the Debate Workspace reveals the Recharts Temporal Evolution chart and the Evidence-Weighted Synthesis card (Figure 3.15), which plot chronological score snapshots from `veritas_db.debate_history` and summarize current PRO, CON, Confidence, and Uncertainty percentages.

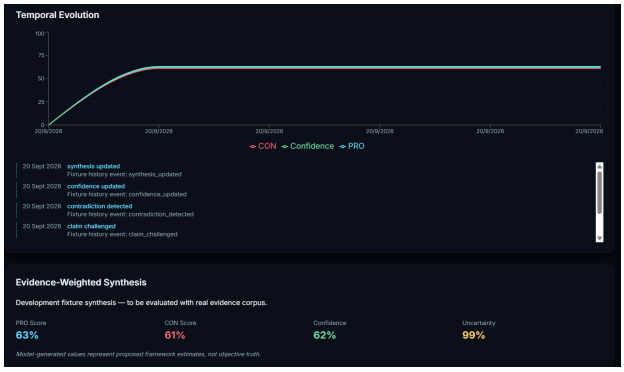


Figure 3.15: Recharts Temporal Evolution timeline (from debate_history) and Evidence-Weighted Synthesis panel showing PRO (63%), CON (61%), Confidence (62%), and Uncertainty (99%).

The Database Health monitor at `/database-health` (Figure 3.16) provides real-time operational verification of the PyMongo connection pool, confirming `CONNECTED` status on `veritas_db`, 8 populated collections, 507 total documents, 16 indexes, and a `2.66 ms` ping latency.

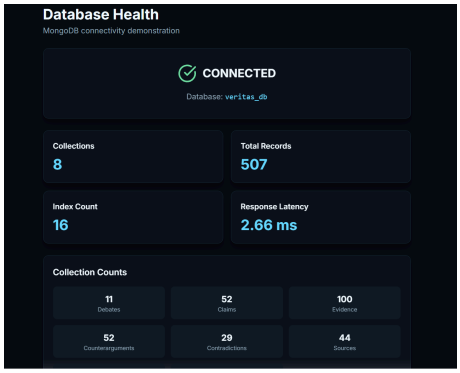


Figure 3.16: Database Health page confirming CONNECTED status on veritas_db, 8 collections, 507 total records, 16 indexes, and 2.66 ms response latency.

3.4 FastAPI Swagger UI CRUD Execution & GitHub Repository Evidence (Rubric B4–B7)

Figures 3.17 through 3.20 document live REST API CRUD execution through FastAPI's OpenAPI Swagger interface (`http://127.0.0.1:8000/docs`), demonstrating `POST /api/debates` (Create), `GET /api/debates/{debate_id}` (Read), `PUT /api/debates/{debate_id}` (Update status to `completed`), and `DELETE /api/debates/{debate_id}` (Cascading Delete).

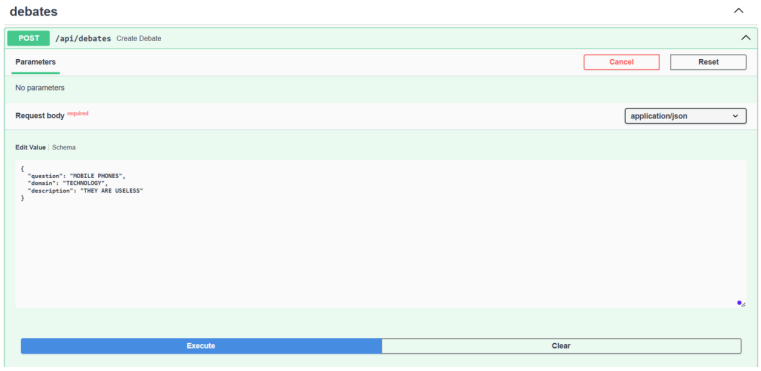


Figure 3.17: CREATE operation — executing POST /api/debates in Swagger UI to insert a new debate document and record a debate_created history event.

After creating a debate via `POST /api/debates`, Figure 3.18 shows the corresponding `GET /api/debates/{debate_id}` read operation retrieving the persisted BSON document by its 24-character hexadecimal `ObjectId`.

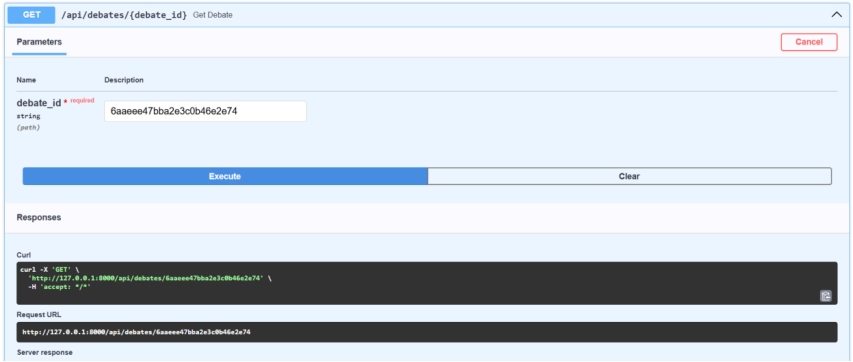


Figure 3.18: READ operation — executing GET /api/debates/{debate_id} for ObjectId 6aaeee47bba2e3c0b46e2e74.

Figure 3.19 demonstrates updating and filtering debate records by domain (`Technology`) and status (`Completed`), verifying that atomic `$set` mutations take effect immediately in MongoDB.

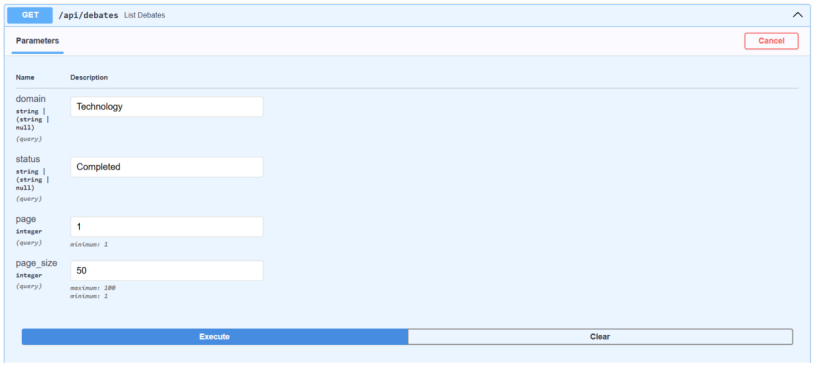


Figure 3.19: UPDATE / FILTER verification — querying completed status updates in Swagger UI.

Finally, Figure 3.20 shows the cascading `DELETE /api/debates/{debate_id}` endpoint removing the disposable test record (`6aaef7c0e6ab4eed455b0053`) and its dependent claims, evidence, counterarguments, and history entries.

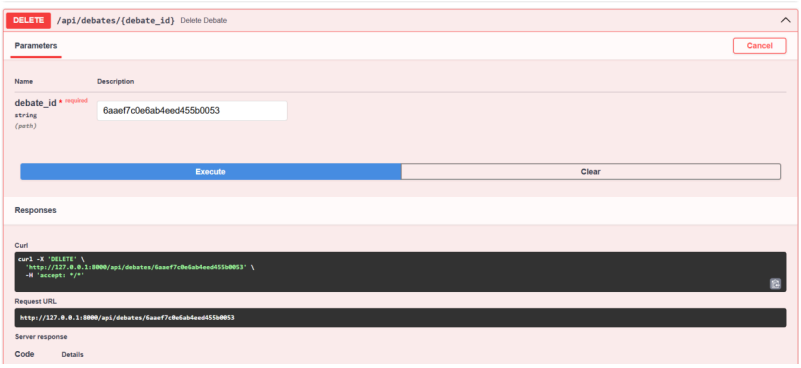


Figure 3.20: DELETE operation — executing DELETE /api/debates/{debate_id} on a test record (6aaef7c0e6ab4eed455b0053).

Figures 3.21 and 3.22 verify the public GitHub repository (https://github.com/siddheshshindeofficial/VERITAS-main) and our command-line Git commit log packaging all 126 source files across backend services, scripts, tests, and React frontend components.

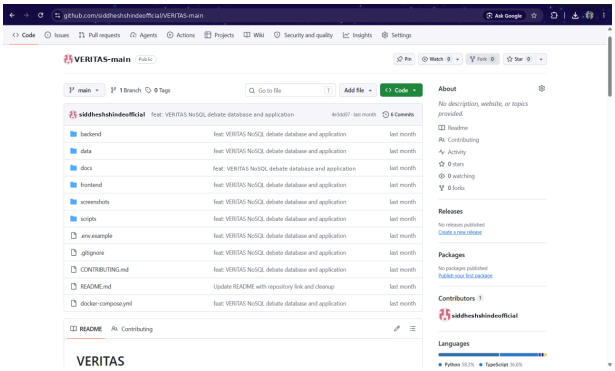


Figure 3.21: Public GitHub repository (https://github.com/siddheshshindeofficial/VERITAS-main) containing backend, frontend, data, scripts, and Docker configuration.

Figure 3.22 captures the PowerShell Git staging and commit output from our development workstation, confirming that all 126 source files (`11,063` lines of Python, TypeScript, and configuration code) were committed cleanly to the `main` branch.

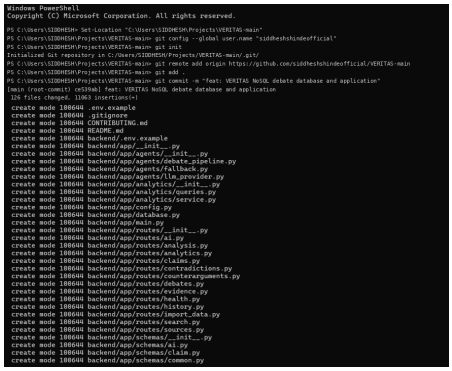


Figure 3.22: Command-line Git initialization and commit log tracking 126 project files across backend and frontend modules.

3.5 DA2 Rubric-to-Implementation Traceability Matrix (Rubric B7)

Rubric	Required Deliverable	Codebase Location	Report Verification
A1	Finalized Problem & Scope	`README.md`, `backend/app/main.py`	Sections 1.1, 2.1, Fig. 3.10
A2	NoSQL Data Model & Schema	`backend/app/schemas/`	Sections 2.2–2.3, Fig. 2.1
A3/B1	Database Implementation	`backend/app/database.py`, `docker-compose.yml`	Figures 3.1–3.5, 3.16
A4	Sample Seeded Dataset	`backend/scripts/seed.py`, `counts.py`	Section 2.3, Figures 3.3–3.5
A5/B3	CRUD Operations	`backend/app/routes/debates.py`, `claims.py`	Section 2.4, Figures 3.17–3.20
A6/B2	Queries Q1–Q12	`backend/app/analytics/queries.py`	Section 2.5, Table 2.2, Figs. 3.6–3.8
A7	Application Indexing	`backend/scripts/indexes.py`	Section 2.6, Figure 3.9
A8	Aggregations (AGG-1 to AGG-5)	`backend/app/analytics/service.py`	Sections 2.5–2.6, Figures 3.7–3.8
A9/B3	App–Database Connectivity	`frontend/src/services/api.ts`, `pages/`	Figures 3.11–3.16
A12/B5	Individual Role & GitHub Link	https://github.com/siddheshshindeofficial/VERITAS-main	Section 2.6, Figures 3.21–3.22

4. Conclusion

In Digital Assignment 2 (Review 2), our team designed, implemented, and verified the complete MongoDB 7 document architecture and full-stack application pipeline for **VERITAS**

(https://github.com/siddheshshindeofficial/VERITAS-main). By combining embedded synthesis state inside root `debates` documents with normalized `ObjectId` references across `claims`, `evidence`, `counterarguments`, `contradictions`, `sources`, `debate_history`, and `embeddings`, the database supports fast single-document workspace reads alongside expressive multi-collection `$lookup` and `$group` aggregations.

As the AI, Scoring Engine, Temporal History, and Visualization Lead (**Siddhesh Shinde, 23BCE5048**), I implemented the four-factor evidence scoring module (`scoring.py`), claim strength and heuristic contradiction detection engines (`claim_strength.py`, `contradiction_engine.py`), append-only temporal history snapshots (`temporal.py`), deterministic fallback synthesis pipeline (`agents/`), and the React 19 user interface integrating React Flow argument graphs and Recharts analytics directly over live FastAPI and PyMongo endpoints.

All twelve project-specific MongoDB queries (`Q1` through `Q12`), five dedicated multi-stage aggregation pipelines (`AGG-1` through `AGG-5`), and eight custom compound and single-field indexes have been verified against live seeded and interactive records in `veritas_db`. Furthermore, our automated 11-test `pytest` suite and live Database Health telemetry confirm sub-3ms local query latency and end-to-end data integrity across the React 19, FastAPI, PyMongo, and MongoDB 7 stack.

Submitted by: Siddhesh Shinde | **Register Number:** 23BCE5048 | **Role:** Member 3 — AI Pipeline, Scoring Engine, Temporal History & Frontend Visualization Lead | **Course:** BCSE406L – NoSQL Databases (Slot D2+TD2, Class CH202670101354) | **Faculty:** Dr. S. Geetha, VIT Chennai.